

1. Enquadramento

Atualmente, o recurso a containers e sistemas de virtualização ultrapassou as soluções tradicionais baseadas em servidores físicos dedicados. Embora os hipervisores de tipo 1 (como o Proxmox VE) constituam a base sólida para a gestão de máquinas virtuais e infraestruturas de rede definidas por software (Software-Defined Network — SDN) em ambientes industriais, corporativos e Edge/Fog Computing, a evolução para arquiteturas de microserviços e dispositivos embebidos exige soluções significativamente mais leves de isolamento e orquestração ao nível do sistema operativo.

Em Sistemas Computacionais Embebidos e plataformas de Edge Computing (ex: gateways industriais, Single-Board Computers como Raspberry Pi ou NVIDIA Jetson), a orquestração tradicional do Kubernetes (k8s) revela-se frequentemente inviável devido ao elevado consumo de memória RAM, CPU e dependência do motor de base de dados distribuída etcd. Nesse cenário, a utilização de distribuições otimizadas e certificadas como o k3s — que condensa todas as funcionalidades do Kubernetes num único binário leve (<100MB) e com baixo consumo de memória (~512MB RAM) — torna-se a solução standard para gerir microserviços em nós com recursos restritos.

Este guião foi estruturado para duas aulas laboratoriais de 2 horas (4 horas no total), articulando a transição do isolamento local com Docker Engine e Docker Compose para a orquestração distribuída e embebida com k3s.

2. Objetivos

Com este trabalho prático de 4 horas (2 aulas × 2 horas) pretende-se que os alunos de Pós-Graduação em Sistemas Computacionais Embebidos desenvolvam competências no aprovisionamento, rede, persistência e orquestração de containers, simulando a interligação entre nós de centro de dados e nós embebidos na infraestrutura Proxmox VE existente. Os objetivos específicos dividem-se em:

- **Aprovisionamento de Ambientes Docker em Proxmox:** Configuração de máquinas virtuais Ubuntu e containers LXC adaptados para Docker Engine e interface de gestão Portainer CE.
- **Exploração de Modos de Rede Standalone:** Implementação e análise prática de topologias de rede em Docker (Bridge, Host e IPVLAN) interligadas às bridges do Proxmox VE (ex: vmbr0).

- **Persistência de Dados em Dispositivos Embebidos:** Configuração e análise das diferenças entre Volumes geridos e Bind Mounts para garantir a persistência de dados em containers efémeros.
- **Aplicações Multi-Container com Docker Compose:** Criação e gestão declarativa de pilhas de serviços (stacks) locais recorrendo a ficheiros docker-compose.yml.
- **Instalação de um Cluster Kubernetes Ligeiro (k3s):** Aprovisionamento do Control Plane e nós de trabalho em instâncias Linux simulando gateways embebidos e servidores Edge.
- **Análise das Otimizações do k3s para Embedded/Edge:** Compreensão do funcionamento do k3s (substituição do etcd por SQLite, binário único compilado em Go, suporte nativo a arquiteturas ARM64/ARMv7 e x86_64).
- **Abstração de Objetos Kubernetes:** Compreensão prática da hierarquia Pod, ReplicaSet, Deployment, Service e Ingress.
- **Gestão Declarativa por Manifestos YAML:** Escrita e aplicação de ficheiros de configuração para deployments de aplicações e exposição de portas.
- **Validação de Auto-Healing e Scaling:** Execução de testes de escalamento dinâmico de réplicas e verificação da recuperação automática de pods eliminados.

3. Desenvolvimento do Trabalho

a. Ambiente de Testes e Requisitos de Infraestrutura

Para a realização das duas atividades práticas, é necessário o acesso ao nó de virtualização Proxmox VE e aos seguintes recursos:

Componente / Recurso	Especificação e Descrição das Instâncias
Hipervisor Host	Proxmox VE (PVE 8.x ou PVE 9.x) com rede vmbr0 ativa.
Instância Principal (VM)	1 VM Ubuntu Server 22.04 LTS (2 vCPU, 2GB RAM, suporte a Docker Engine, Compose e k3s).
Instância Secundária (LXC)	1 Container LXC Ubuntu (1 vCPU, 1GB RAM) simulando um nó Edge com opções nesting=1 e keyctl=1 ativadas no Proxmox VE.
Credenciais e Ferramentas	Credenciais: user: root / passwd: 12345 Ferramentas: Docker Engine,

	Docker Compose, Portainer CE e k3s / kubectl CLI.
--	---

a. AULA 1: Docker Standalone, Redes Virtuais, Volumes e Docker Compose

Nesta primeira fase, instala-se o Docker Engine e a ferramenta Portainer CE para gestão gráfica, explorando de seguida os três tipos fundamentais de redes num container standalone. Este servidor pode ser baseado num LXC com 512M, 2 core, ubuntu 22.04

Para executar o Docker dentro de um Container LXC no Proxmox VE, é obrigatório aceder a Proxmox Web UI -> LXC -> Options -> Features e ativar as opções Nesting (nesting=1) e Keyctl (keyctl=1). Sem estas opções, o motor de containers falhará a criação de namespaces no kernel do Host.

```
# 1. Atualização e Instalação do Docker Engine, Plugin Compose e Dependências (via Shell)
apt update && apt install -y curl ca-certificates gnupg lsb-release
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-
keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | tee
/etc/apt/sources.list.d/docker.list > /dev/null
apt update && apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# 2. Instalação da Interface de Gestão Portainer CE (na VM Ubuntu)
docker volume create portainer_data
docker run -d -p 9000:9000 --name=portainer --restart=always -v
/var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-
ce:latest
```

Configuração e Teste Prático dos Três Modos de Rede Standalone:

Modo de Rede	Comando de Deploy e Teste	Comportamento Operacional de Rede
Rede Bridge	docker run -d --name web_bridge -p 8081:80 nginx	O servidor web é isolado numa ponte interna do Docker e interligado por NAT/SNAT à bridge vmbr0 do Proxmox.

Rede Host	<code>docker run -d --name web_host --net=host nginx</code>	O container remove o isolamento de rede, expondo os seus sockets diretamente nas interfaces do SO e na bridge vmbr0.
Rede IPVLAN	<code>docker network create -d ipvlan --subnet=192.168.1.0/24 --gateway=192.168.1.1 -o parent=eth0 net_ipvlan</code> <code>docker run -d --name web_ipvlan --net=net_ipvlan --ip=192.168.1.150 nginx</code>	Cria uma sub-interface L2/L3 associada à eth0, atribuindo um IP diretamente acessível na rede física/virtual sem necessidade de NAT.

A imutabilidade dos containers exige mecanismos explícitos para guardar dados. Nesta etapa, exploram-se os Volumes e cria-se uma aplicação multi-container gerida declarativamente pelo Docker Compose.

i. Teste de Persistência com Volumes e Bind Mounts:

```
# 1. Criar um Volume gerido pelo Docker Engine
docker volume create dados_web

# 2. Arrancar container associando o volume à diretoria do Nginx
docker run -d --name web_persistent -v dados_web:/usr/share/nginx/html -p 8082:80 nginx

# 3. Alterar o conteúdo index.html dentro do volume
docker exec -it web_persistent sh -c "echo '<h1>AULA DOCKER PG - DADOS PERSISTENTES</h1>' > /usr/share/nginx/html/index.html"

# 4. Remover o container forçadamente e recriar outro associado ao mesmo volume
docker rm -f web_persistent
docker run -d --name web_persistent2 -v dados_web:/usr/share/nginx/html -p 8082:80 nginx

# 5. Validação: Aceder ao porto 8082 e comprovar que os dados sobreviveram à destruição do container
curl http://localhost:8082
```

ii. Orquestração Multi-Container com Docker Compose:

Crie uma diretoria de trabalho /opt/app-stack e defina o ficheiro docker-compose.yml:

```
version: '3.8'

services:
  web:
    image: nginx:1.25-alpine
    container_name: web_compose
    ports:
      - "8080:80"
    volumes:
      - html_data:/usr/share/nginx/html
    networks:
      - app_net
    depends_on:
      - db

  db:
    image: redis:7-alpine
    container_name: redis_compose
    volumes:
      - redis_data:/data
    networks:
      - app_net

volumes:
  html_data:
  redis_data:

networks:
  app_net:
    driver: bridge
```

iii. Comandos de Gestão do Lifecycle com Docker Compose CLI:

```
# 1. Arrancar a pilha de serviços em background
docker compose up -d

# 2. Verificar o estado dos containers e redes criadas pelo Compose
docker compose ps
docker compose logs -f web

# 3. Validar a comunicação interna entre o container web e o container redis na rede bridge dedicada
docker exec -it web_compose ping db -c 2

# 4. Parar e remover a pilha de serviços preservando os volumes
docker compose down
```

b. AULA 2: Introdução Prática ao Kubernetes para Sistemas Embebidos (k3s)

Na segunda aula, introduz-se a orquestração distribuída recorrendo ao k3s, uma distribuição de Kubernetes totalmente certificada e desenvolvida pela Rancher/SUSE focada em Sistemas Embebidos, IoT Gateways e Edge Computing.

Porquê o k3s para Sistemas Computacionais Embebidos? (Análise Técnica PG):

- **Pegada de Memória Mínima (Low Footprint):** Redução do consumo de RAM de ~2GB (k8s tradicional) para ~512MB RAM no k3s, permitindo a execução em Single-Board Computers (SBCs) ou instâncias LXC com recursos reduzidos.
- **Binário Único e Simplificado:** Todo o Control Plane (kube-apiserver, kube-scheduler, kube-controller-manager) e o Kubelet estão integrados num único ficheiro executável em Go com menos de 100MB.
- **Substituição do etcd por SQLite/kine:** Por predefinição em nós isolados ou de pequenas dimensões, o k3s substitui a base de dados distribuída etcd (com elevadas exigências de I/O em disco) por um motor SQLite interno leve.
- **Suporte Multi-Arquitetura NATIVO:** Compilado nativamente para ARM64 e ARMv7 (arquiteturas dominantes em sistemas embebidos e IoT) bem como x86_64.
- **Baterias Incluídas:** Inclui nativamente Containerd (runtime), Flannel (CNI de rede overlay leve), CoreDNS, Traefik (Ingress Controller) e ServiceLB (Klipper LoadBalancer).

i. Instalação e Validação do k3s no Nó Master (VM Ubuntu):

```
# 1. Instalar o k3s em modo Single-Node / Control Plane (na VM Ubuntu)
curl -sL https://get.k3s.io | SH_K3S_EXEC="--write-kubeconfig-mode 644" sh -

# 2. Configurar a variável KUBECONFIG para acesso via kubectl
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
kubectl get nodes -o wide

# 3. Verificar a pegada e os pods de sistema leves no namespace kube-system
kubectl get pods -n kube-system
```

A gestão de workloads no k3s segue rigorosamente a especificação standard da API do Kubernetes através de manifestos declarativos em formato YAML.

ii. Criação do Manifesto de Aplicação e Serviço:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: web-pg
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-pg
  template:
    metadata:
      labels:
        app: web-pg
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
spec:
  type: NodePort
  selector:
    app: web-pg
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

iii. Execução, Escalamento e Teste de Auto-Healing via CLI:

```
# 1. Aplicar o manifesto declarativo no Cluster Kubernetes k3s
kubectl apply -f app-demo.yaml

# 2. Inspeccionar o estado dos Deployments, Pods e Services criados
kubectl get deployments
kubectl get pods -l app=web-pg -o wide
kubectl get svc nginx-nodeport-service

# 3. Teste de Escalamento Dinâmico: Aumentar o número de réplicas para 5
kubectl scale deployment nginx-deployment --replicas=5
kubectl get pods -l app=web-pg
```

```
# 4. Teste de Auto-Healing (Recuperação Autónoma): Forçar a eliminação manual de um Pod
kubectl delete pod <NOME_DE_UM_DOS_PODS>

# 5. Validação imediata: Observar que o Kubernetes cria instantaneamente um novo Pod para
manter o estado desejado (5 réplicas)
kubectl get pods -l app=web-pg

# 6. Testar o acesso externo ao serviço exposto via NodePort na porta 30080
curl http://192.168.1.50:30080
```

c. Bateria de Testes, Diagnóstico e Validação Prática

Para garantir o correto diagnóstico das infraestruturas em ambas as aulas, os alunos devem executar e documentar a seguinte sequência de comandos de verificação:

```
# =====
# BATERIA DE DIAGNÓSTICO AULA 1: DOCKER STANDALONE, VOLUMES & DOCKER COMPOSE
# =====
docker info                # Inspeccionar o estado geral do motor Docker
docker network ls          # Listar redes ativas (bridge, host, ipvlan)
docker volume ls           # Listar volumes de persistência criados no host
docker compose ps          # Verificar estado dos serviços geridos pelo Docker
Compose
docker container inspect web_bridge # Analisar o mapeamento de IP e gateway interno

# =====
# BATERIA DE DIAGNÓSTICO AULA 2: KUBERNETES LIGEIRO (K3S)
# =====
kubectl cluster-info        # Validar o estado do Control Plane e serviços de rede
kubectl get events --sort-by='.metadata.creationTimestamp' # Registos de auditoria e
escalamento
kubectl describe svc nginx-nodeport-service # Inspeccionar os endpoints IPs associados ao
Serviço
kubectl logs -l app=web-pg --tail=20 # Inspeccionar os logs agregados dos Pods em execução
```

4. Avaliação e Questões de Reflexão Crítica

A avaliação será realizada em ambiente laboratorial através da demonstração prática das topologias a funcionar e da apresentação de um relatório técnico com evidências:

Critério de Avaliação	Descrição e Evidências Demonstrativas
Configuração Docker & Redes	Demonstração funcional das redes Bridge, Host e IPVLAN e integração gráfica no Portainer CE.

Persistência & Compose	Validação de volumes de dados e arranque/gestão de pilhas multi-container via docker compose.
Orquestração k3s em Embedded/Edge	Aplicação com sucesso do manifesto YAML, escalamento de réplicas e exposição via NodePort em k3s.
Análise Crítica e Relatório	Resposta fundamentada às questões de reflexão de nível avançado sobre arquiteturas de containers e sistemas embebidos.

5. Questões de Reflexão Crítica

As questões seguintes devem ser respondidas no relatório laboratorial para consolidar a fundamentação teórica:

Questão 1 (Virtualização por Hypervisor vs. Containers & Modos de Rede em Edge Computing):

Compare a arquitetura de isolamento e sobreposição (overhead) de recursos de uma VM KVM no Proxmox VE em relação a um Container Docker executado num gateway embebido. Na sua resposta, analise de que forma a escolha de uma rede IPVLAN (estudada na Aula 1) reduz a latência de processamento em comparação com as pontes virtuais convencionais com NAT (vbr0 / docker0) em cenários de recolha de dados de sensores.

Questão 2 (Arquitetura k3s vs. k8s Tradicional em Dispositivos Embebidos):

Explique quais as principais alterações arquiteturais introduzidas na distribuição k3s que viabilizam a sua execução em sistemas embebidos e Single-Board Computers (SBCs) com recursos computacionais restritos. Na sua resposta, compare a utilização do SQLite (no k3s) em relação ao etcd (no Kubernetes tradicional) no que concerne ao consumo de memória RAM, operações de escrita/leitura em cartão SD/eMMC e complexidade de manutenção do cluster.

Questão 3 (Integração de Infraestruturas: SDN Proxmox VE com CNI Flannel no k3s):

Analise como a infraestrutura de Software-Defined Network (SDN) do Proxmox VE 8/9 (com VNETs isoladas, zonas L2/L3 e regras de firewall nftables) pode ser combinada com o Container Network Interface (CNI) Flannel nativo do k3s para garantir a comunicação segura e o isolamento multitenant entre serviços de centros de dados centrais e gateways embebidos na periferia (Edge).

6. Recursos

- **Credenciais padrão para LXC/VMs de teste:** user: root | passwd: 12345
- **Documentação Oficial Docker Engine e Compose:** <https://docs.docker.com>
- **Instalação do Portainer CE em Linux:** <https://docs.portainer.io>
- **Documentação Oficial k3s (Lightweight Kubernetes for Embedded/Edge):** <https://docs.k3s.io>
- **Documentação Oficial Kubernetes (Tasks & Concepts):** <https://kubernetes.io/docs/home/>
- **Proxmox VE SDN Wiki e Guia de Redes:** https://pve.proxmox.com/wiki/Software-Defined_Network